Design and Development of an Intelligent Real-Time Traffic Congestion Prediction System Using Machine Learning

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

ITA0606-Machine Learning

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY IN

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

M. Vidhya (192412151)
C. Sai Harshitha(192424346)
K. Dhanaswini (192421326)

Under the Supervision of

Dr. K. Baskar

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical

Sciences Chennai-602105

SEPTEMBER 2026

ITA0606 - Machine Learning

Assignment Question

Course Code & Name	ITA0614 - Machine Learning
Assignment Title	Design and Development of an Intelligent Real-Time Traffic Congestion Prediction System Using Machine Learning
Course Outcome(s) - CO	**CO6** : Applies Decision Tree Learning for representing and classifying traffic-congestion concepts. **CO7**: Uses NumPy and Pandas for data preprocessing, statistical analysis, aggregation, and visualization. **CO8**: Integration and application of theoretical and practical ML knowledge
Bloom's Taxonomy Level	Apply (L3) + Analyze (L4)
SDG Mapping (SDG 1- 17)	SDG 11 – Sustainable Cities and Communities SDG 9 – Industry, Innovation and Infrastructure

1. PROBLEM STATEMENT AND PROBLEM FORMULATION

1.1 Problem Statement

Traffic congestion is one of the major challenges faced by rapidly developing cities. The continuous increase in the number of vehicles, limited road capacity, improper traffic management, peak-hour traffic, and unpredictable traffic conditions contribute to congestion. Heavy traffic results in increased travel time, fuel consumption, air pollution, and inconvenience to commuters.

Conventional traffic monitoring systems mainly focus on observing existing traffic conditions. However, simply monitoring the current traffic condition may not be sufficient for effective traffic management. A system that can automatically analyze traffic information and predict the congestion level can provide useful information for traffic monitoring and planning.

Machine Learning provides an effective approach for analyzing historical traffic data and identifying patterns between traffic parameters and congestion levels. In this project, a

Decision Tree Classifier is developed to classify traffic conditions into **Low, Medium, and High congestion**.

The proposed system uses measurable traffic parameters such as **vehicle count, average vehicle speed, and time of day** as input features. The Decision Tree learns the relationship between these features and the congestion class from historical data. After training, the model can predict the congestion level for new traffic conditions.

1.2 Problem Formulation

The problem is formulated as a **supervised multi-class classification problem**.

Let the input feature vector be:

X = {Vehicle Count, Average Speed, Time of Day}

The target variable is:

Y = {Low, Medium, High}

The objective is to develop a classification model:

f(X) → Y

where the model receives traffic-related input parameters and predicts the corresponding congestion level.

The dataset is divided into training and testing portions. The training dataset is used to construct the Decision Tree, while the testing dataset is used to evaluate the model's ability to classify previously unseen traffic conditions.

2. OBJECTIVE AND EXPECTED OUTCOMES

2.1 Objectives

The main objective of this project is to design and develop an intelligent Machine Learning-based system for traffic congestion prediction.

The specific objectives are:

1. To study the causes and characteristics of traffic congestion.
2. To identify important parameters that influence congestion.
3. To collect or obtain suitable traffic data for model development.
4. To preprocess and clean the traffic dataset.
5. To select relevant input features such as vehicle count, average speed, and time of day.
6. To formulate congestion prediction as a classification problem.
7. To implement a **Decision Tree Classifier**.
8. To use **Gini Impurity** as the splitting criterion.
9. To train the model using historical traffic data.

10. To test the model using unseen data.
11. To classify traffic conditions into Low, Medium, and High congestion.
12. To evaluate the model using accuracy, precision, recall, F1-score, and confusion matrix.
13. To visualize the prediction and performance results.
14. To analyze the strengths and limitations of the developed model.

2.2 Expected Outcomes

The expected outcomes of the project are:

- A properly prepared traffic dataset.
- Identification of relevant traffic features.
- A trained Decision Tree classification model.
- Prediction of three congestion levels.
- Performance evaluation of the model.
- Graphical representation of traffic and prediction results.
- A simple and interpretable Machine Learning solution for traffic congestion monitoring.

3. REQUIREMENTS, CONSTRAINTS AND ASSUMPTIONS

3.1 Hardware Requirements

- Laptop/Desktop computer
- Minimum 4 GB RAM
- Dual-core or higher processor
- Minimum 10 GB available storage
- Internet connection for dataset collection and software installation

3.2 Software Requirements

- Python 3.x
- Google Colab or Jupyter Notebook
- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- Seaborn
- Web browser

3.3 Functional Requirements

The system should be able to:

1. Accept traffic-related data as input.
2. Process and clean the input data.
3. Select relevant traffic features.
4. Train the Decision Tree model.
5. Predict the congestion level.
6. Display prediction results.
7. Calculate model performance metrics.
8. Generate graphs and a confusion matrix.

3.4 Non-Functional Requirements

- **Accuracy:** The system should provide reliable predictions.
- **Efficiency:** Model training and prediction should require reasonable computation time.
- **Usability:** The output should be easy to understand.
- **Scalability:** The system should allow additional traffic features in future.
- **Maintainability:** The model and code should be easy to modify.
- **Interpretability:** Decision Tree rules should be understandable.

3.5 Constraints

1. The model performance depends on the quality of the dataset.
2. Limited traffic features may affect prediction accuracy.
3. Sudden accidents and road closures may not be represented in historical data.
4. The basic system may not use live sensor data.
5. Real-time operation requires continuously updated traffic information.
6. A Decision Tree may overfit if it becomes excessively deep.

3.6 Assumptions

1. Vehicle count data is available for the selected road or location.
2. Average speed is available or can be calculated.
3. The historical dataset contains representative traffic conditions.
4. Congestion labels in the dataset are correctly defined.
5. Vehicle count and average speed have a significant relationship with congestion.
6. Traffic patterns are reasonably consistent during similar time periods.

4. APPLICATION OF RELEVANT COURSE KNOWLEDGE / CONCEPTS

This project applies several concepts from Machine Learning and data analysis.

4.1 Supervised Learning

The project uses supervised learning because the training dataset contains input features and corresponding congestion labels.

4.2 Classification

The output consists of discrete classes:

- Low
- Medium
- High

Therefore, the problem is treated as a classification problem.

4.3 Decision Tree

A Decision Tree represents classification decisions in a tree structure consisting of:

- Root node
- Internal decision nodes
- Branches
- Leaf nodes

Each internal node represents a decision based on an input feature, while the leaf node represents the final congestion class.

4.4 Gini Impurity

Gini Impurity is used to measure the impurity of a node. The Decision Tree selects splits that produce purer groups of data.

The Gini impurity is:

$$\mathbf{Gini = 1 - \Sigma\, p_i^2}$$

where $\mathbf{p_i}$ represents the proportion of samples belonging to class *i*.

4.5 Data Preprocessing

Data preprocessing involves:

- Removing duplicate records.
- Handling missing values.
- Checking incorrect values.
- Converting data into suitable formats.
- Selecting relevant features.

4.6 Train-Test Split

The dataset is divided into training and testing data. The training data is used to build the model, while the testing data evaluates its performance.

4.7 Performance Evaluation

The developed model is evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

5. DESIGN / PROPOSED SOLUTION / METHODOLOGY

5.1 Proposed System

The proposed system consists of several stages. Traffic data is first collected from a suitable dataset. The data is then preprocessed to remove errors and missing values.

Relevant traffic parameters are selected as input features. The dataset is divided into training and testing sets. A Decision Tree Classifier is trained using the training data with Gini Impurity as the splitting criterion.

5.2 Input Parameters

Vehicle Count: Number of vehicles observed during a particular period.

Average Speed: Average speed of vehicles travelling through the selected road segment.

Time of Day: Time information used to identify different traffic periods such as peak and non-peak hours.

5.3 Output

The system provides one of the following:

- **Low Congestion**
- **Medium Congestion**
- **High Congestion**

5.4 Overall Methodology

Step 1: Traffic Dataset Collection

↓

Step 2: Data Cleaning

↓

Step 3: Feature Selection

↓

Step 4: Target Label Identification

↓

Step 5: Train-Test Split

↓

Step 6: Decision Tree Model Development

↓

Step 7: Gini-Based Splitting

↓

Step 8: Model Training

↓

Step 9: Prediction

↓

Step 10: Performance Evaluation

↓

Step 11: Visualization and Analysis

6. ALGORITHM / PSEUDOCODE / FLOWCHART

6.1 Decision Tree Algorithm

1. Start.
2. Load the traffic dataset.
3. Examine the dataset.
4. Remove missing and invalid values.
5. Select vehicle count, average speed, and time of day.
6. Select congestion level as the target.
7. Divide the dataset into training and testing data.
8. Initialize the Decision Tree Classifier.
9. Set the splitting criterion to Gini.
10. Train the classifier.
11. Provide test data to the trained model.
12. Generate predicted congestion classes.
13. Compare predicted values with actual values.
14. Calculate performance metrics.
15. Display results.
16. Stop.

6.2 Pseudocode

START

Load traffic dataset

Check missing values and duplicate records

Clean and preprocess the dataset

Select input features:

- Vehicle Count
- Average Speed
- Time of Day

Select target:

- Congestion Level

Divide dataset into:

- Training Data
- Testing Data

Create Decision Tree Classifier

Set splitting criterion = Gini

Train the model using training data

Use testing data for prediction

Predict:

- Low / Medium / High

Calculate:

- Accuracy
- Precision
- Recall
- F1-score

Generate confusion matrix

Display graphs and results

END

7. IMPLEMENTATION / SOURCE CODE AND ENVIRONMENT / TOOLS USED

7.1 Programming Language

The project is implemented using Python because Python provides extensive libraries for Machine Learning, data processing, and visualization.

7.2 Libraries Used

```
import numpy as np
import pandas as pd
import matplotlib. pyplot as plt
```

```
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
data = pd.read_csv("traffic_data.csv")
print(data.head())
data.info()
print("\nDataset Shape:", data.shape)
print(data.isnull().sum())
categorical_cols = ["Time_of_Day", "Day_of_Week", "Weather"]
for col in categorical_cols:
    data[col] = data[col].astype("string").str.strip().str.title()
numeric_cols = [
    "Vehicle_Count",
    "Average_Speed",
    "Road_Occupancy"
]
for col in numeric_cols:
    data[col] = data[col].fillna(data[col].median())
for col in categorical_cols:
    data[col] = data[col].fillna(data[col].mode()[0])
print(data.isnull().sum())
print("Duplicate rows before removal:", data.duplicated().sum())
data = data.drop_duplicates()
print("Duplicate rows after removal:", data.duplicated().sum())
print("Dataset shape after cleaning:", data.shape)
print(data.describe())
```

```
group_analysis = data.groupby("Congestion_Level")[
    ["Vehicle_Count", "Average_Speed", "Road_Occupancy"]
].mean()
print(group_analysis)
numeric_data = data.select_dtypes(include=np.number)
correlation = numeric_data.corr()
print(correlation)
plt.figure(figsize=(8, 6))
sns.heatmap(correlation, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
plt.figure(figsize=(7, 5))
sns.countplot(data=data, x="Congestion_Level")
plt.title("Distribution of Congestion Levels")
plt.xlabel("Congestion Level")
plt.ylabel("Number of Records")
plt.tight_layout()
plt.show()
plt.figure(figsize=(8, 5))
sns.boxplot(data=data, x="Congestion_Level", y="Vehicle_Count")
plt.title("Vehicle Count vs Congestion Level")
plt.xlabel("Congestion Level")
plt.ylabel("Vehicle Count")
plt.tight_layout()
plt.show()
plt.figure(figsize=(8, 5))
sns.boxplot(data=data, x="Congestion_Level", y="Average_Speed")
plt.title("Average Speed vs Congestion Level")
plt.xlabel("Congestion Level")
plt.ylabel("Average Speed")
plt.tight_layout()
plt.show()
plt.figure(figsize=(8, 5))
```

```
sns.boxplot(data=data, x="Congestion_Level", y="Road_Occupancy")
plt.title("Road Occupancy vs Congestion Level")
plt.xlabel("Congestion Level")
plt.ylabel("Road Occupancy (%)")
plt.tight_layout()
plt.show()
plt.figure(figsize=(8, 5))
sns.countplot(
    data=data,
    x="Weather",
    hue="Congestion_Level"
)
plt.title("Weather vs Congestion Level")
plt.xlabel("Weather")
plt.ylabel("Number of Records")
plt.tight_layout()
plt.show()
plt.figure(figsize=(8, 5))
sns.countplot(
    data=data,
    x="Time_of_Day",
    hue="Congestion_Level"
)
plt.title("Time of Day vs Congestion Level")
plt.xlabel("Time of Day")
plt.ylabel("Number of Records")
plt.tight_layout()
plt.show()
data_encoded = pd.get_dummies(
    data,
    columns=["Weather", "Day_of_Week", "Time_of_Day"],
    drop_first=True
)
print("\nEncoded dataset:")
```

```
print(data_encoded.head())
X = data_encoded.drop("Congestion_Level", axis=1)
y = data_encoded["Congestion_Level"]
print("Input features:")
print(list(X.columns))
print("\nTarget variable: Congestion_Level")
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=5,
    random_state=42
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Predictions:")
print(y_pred)
plt.figure(figsize=(20, 10))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=model.classes_,
    filled=True,
    rounded=True
)
plt.title("Decision Tree for Traffic Congestion Prediction")
plt.tight_layout()
```

```
plt.show()
importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})
importance = importance.sort_values(
    by="Importance",
    ascending=False
)
print("\nFeature Importance:")
print(importance)
plt.figure(figsize=(10, 6))
sns.barplot(
    data=importance,
    x="Importance",
    y="Feature"
)
plt.title("Decision Tree Feature Importance")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", round(accuracy * 100, 2), "%")
precision = precision_score(
    y_test,
    y_pred,
    average="weighted",
    zero_division=0
)
print("Precision:", round(precision, 4))
recall = recall_score(
    y_test,
    y_pred,
```

```
    average="weighted",
    zero_division=0
)
print("Recall:", round(recall, 4))
f1 = f1_score(
    y_test,
    y_pred,
    average="weighted",
    zero_division=0
)
print("F1-Score:", round(f1, 4))
print(
    classification_report(
        y_test,
        y_pred,
        zero_division=0
    )
)
cm = confusion_matrix(y_test, y_pred)
print(cm)
plt.figure(figsize=(7, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    xticklabels=model.classes_,
    yticklabels=model.classes_
)
plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.tight_layout()
plt.show()
new_data = pd.DataFrame({
```

```
    "Vehicle_Count": [110],
    "Average_Speed": [22],
    "Time_of_Day": ["Evening"],
    "Day_of_Week": ["Friday"],
    "Weather": ["Rainy"],
    "Road_Occupancy": [82]
})
print("\nNew traffic data:")
print(new_data)
for col in categorical_cols:
    new_data[col] = new_data[col].astype("string").str.strip().str.title()
new_data_encoded = pd.get_dummies(
    new_data,
    columns=["Weather", "Day_of_Week", "Time_of_Day"],
    drop_first=True)
new_data_encoded = new_data_encoded.reindex(
    columns=X.columns,
    fill_value=0)
prediction = model.predict(new_data_encoded)
print(
    "\nPredicted Congestion Level:",
    prediction[0])
if prediction[0] == "Low":
    recommendation = (
        "Maintain normal signal timing and continue routine monitoring."
    )
elif prediction[0] == "Medium":
    recommendation = (
        "Adjust signal timing, monitor traffic density and consider "
        "alternative routes." )
else:
    recommendation = (
        "Increase green-light duration on congested routes, divert traffic "
        "where possible and issue congestion alerts."
```

```
    )
print("\nRecommended Traffic Action:")
print(recommendation)
print("Maximum Depth:", model.max_depth)
print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))
print("Accuracy:", round(accuracy * 100, 2), "%")
print("Precision:", round(precision, 4))
print("Recall:", round(recall, 4))
print("F1-Score:", round(f1, 4))
print("Real-Time Prediction:", prediction[0])
print("\nTraffic congestion prediction process completed successfully.")
```

7.3 Model Configuration

Algorithm: Decision Tree Classifier

Criterion: Gini Impurity

Task: Multi-class Classification

Target: Traffic Congestion Level

Classes: Low, Medium, High

8.**TEST CASES AND EXPECTED / ACTUAL RESULTS**

Test Case	**Vehicle Count**	**Average Speed**	**Time**	**Expected**	**Actual**
TC01	Low	High	Off-Peak	Low	Obtain from model
TC02	Medium	Medium	Normal	Medium	Obtain from model
TC03	High	Low	Peak	High	Obtain from model
TC04	Very High	Very Low	Peak	High	Obtain from model
TC05	Low	High	Off-Peak	Low	Obtain from model

9. EXECUTION SCREENSHOTS / OUTPUTS

Fig no 9.1: DATA LOADING

```
First 5 rows:
   Vehicle_Count  Average_Speed Time_of_Day Day_of_Week Weather
0           35.0           52.0     Morning      Monday   Clear
1           42.0           49.0     Morning     Tuesday   Clear
2           50.0           46.0     Morning   Wednesday   Clear
3           58.0           43.0     Morning    Thursday  Cloudy
4           65.0           40.0     Morning      Friday   Clear

   Road_Occupancy Congestion_Level
0              25              Low
1              30              Low
2              35              Low
3              40           Medium
4              45           Medium
```

Fig no 9.2: MISSING VALUE ANALYSIS

```
Missing values before preprocessing:
Vehicle_Count       1
Average_Speed       2
Time_of_Day         1
Day_of_Week         1
Weather             1
Road_Occupancy      0
Congestion_Level    0
dtype: int64
```

```
--- Numerical Missing Values ---
Vehicle_Count: 1 missing value(s) -> Replaced using median = 70.00
Average_Speed: 2 missing value(s) -> Replaced using median = 38.00
Road_Occupancy: No missing values

--- Categorical Missing Values ---
Time_of_Day: 1 missing value(s) -> Replaced using mode = Evening
Day_of_Week: 1 missing value(s) -> Replaced using mode = Monday
Weather: 1 missing value(s) -> Replaced using mode = Clear
```

Fig no 9.3: DUPLICATE REMOVAL

```
Duplicate rows before removal: 0
Duplicate rows after removal: 0
Dataset shape after cleaning: (51, 7)
```

```
Duplicate handling method: Exact duplicate rows removed.


Duplicate records after removal: 0
Dataset shape after duplicate removal: (50, 7)
```

Fig no 9.4: STATISTICAL ANALYSIS

```
Descriptive Statistics:
       Vehicle_Count  Average_Speed  Road_Occupancy
count      51.000000      51.000000       51.000000
mean       73.156863      36.254902       52.058824
std        24.921776      10.374667       18.714071
min        30.000000      18.000000       20.000000
25%        53.500000      27.000000       37.000000
50%        73.000000      37.000000       51.000000
75%        95.000000      44.500000       68.000000
max       120.000000      55.000000       88.000000
```

Fig no 9.5: GROUP-BY ANALYSIS

```
Average traffic characteristics by congestion level:
                  Vehicle_Count  Average_Speed  Road_Occupancy
Congestion_Level
High                 102.750000      23.937500       74.750000
Low                   42.857143      49.071429       29.642857
Medium                70.809524      37.095238       49.714286
```

Fig no 9.6: CORRELATION ANALYSIS

```
Correlation Matrix:
                Vehicle_Count  Average_Speed  Road_Occupancy
Vehicle_Count        1.000000      -0.994375        0.998762
Average_Speed       -0.994375       1.000000       -0.994968
Road_Occupancy       0.998762      -0.994968        1.000000
```

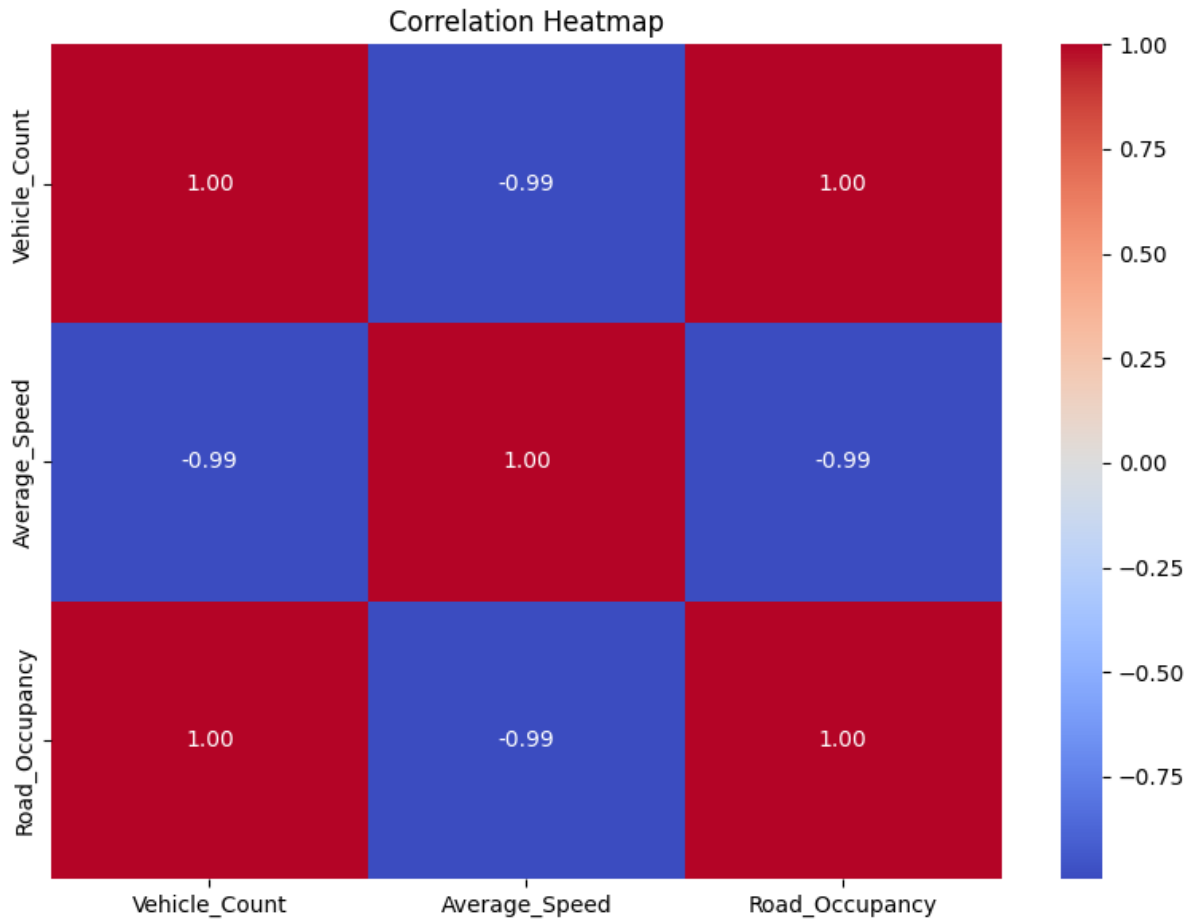


Fig no 9.7: VISUALIZATION 1 – CONGESTION DISTRIBUTION

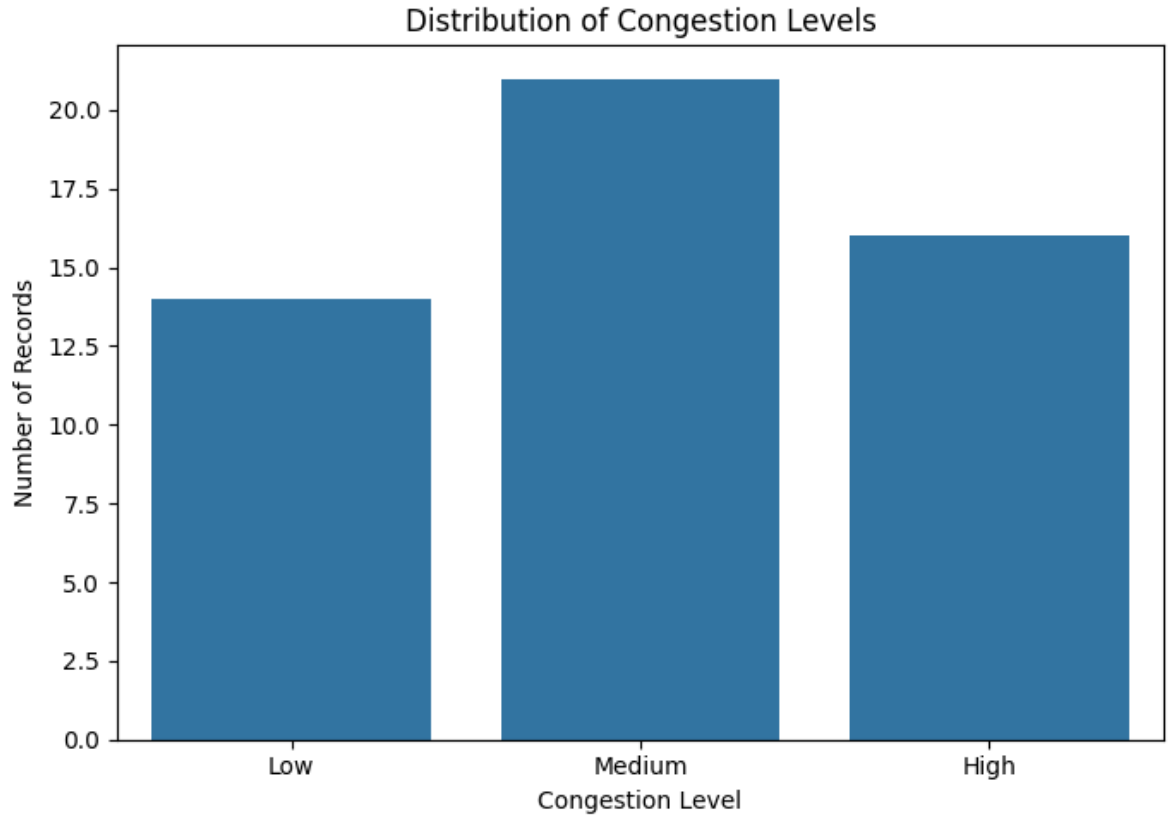


Fig no 9.8: VISUALIZATION 2 – VEHICLE COUNT VS CONGESTION

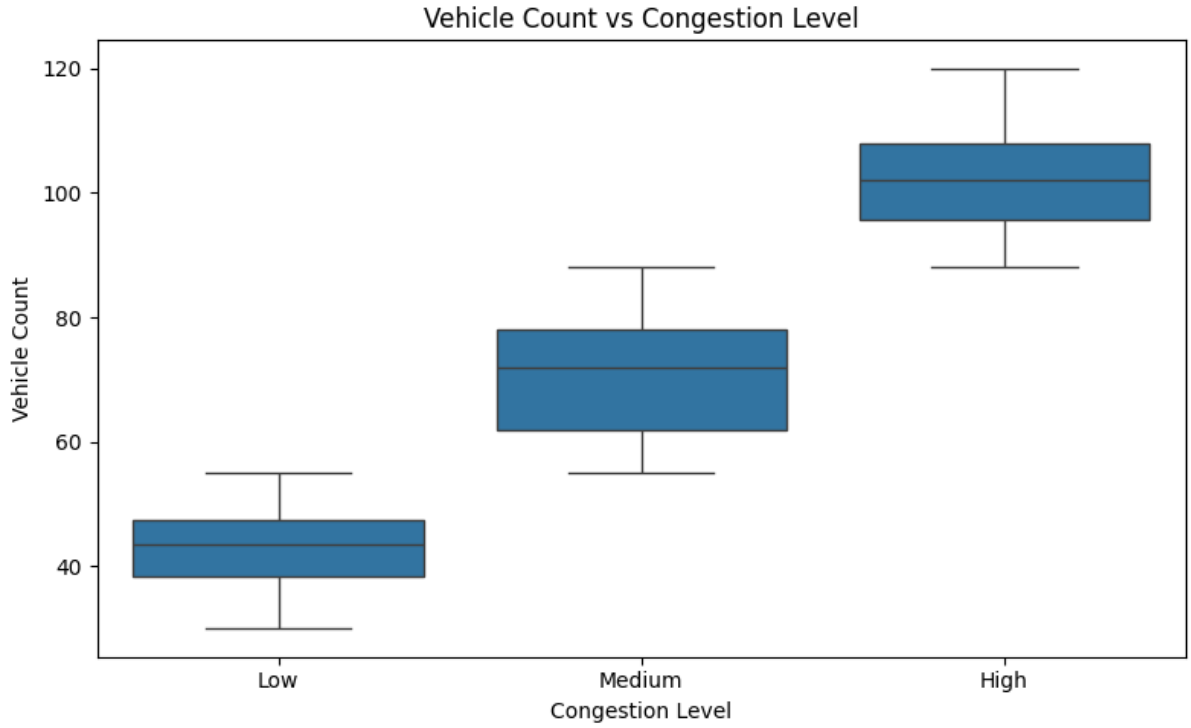


Fig no 9.9: VISUALIZATION 3 – SPEED VS CONGESTION

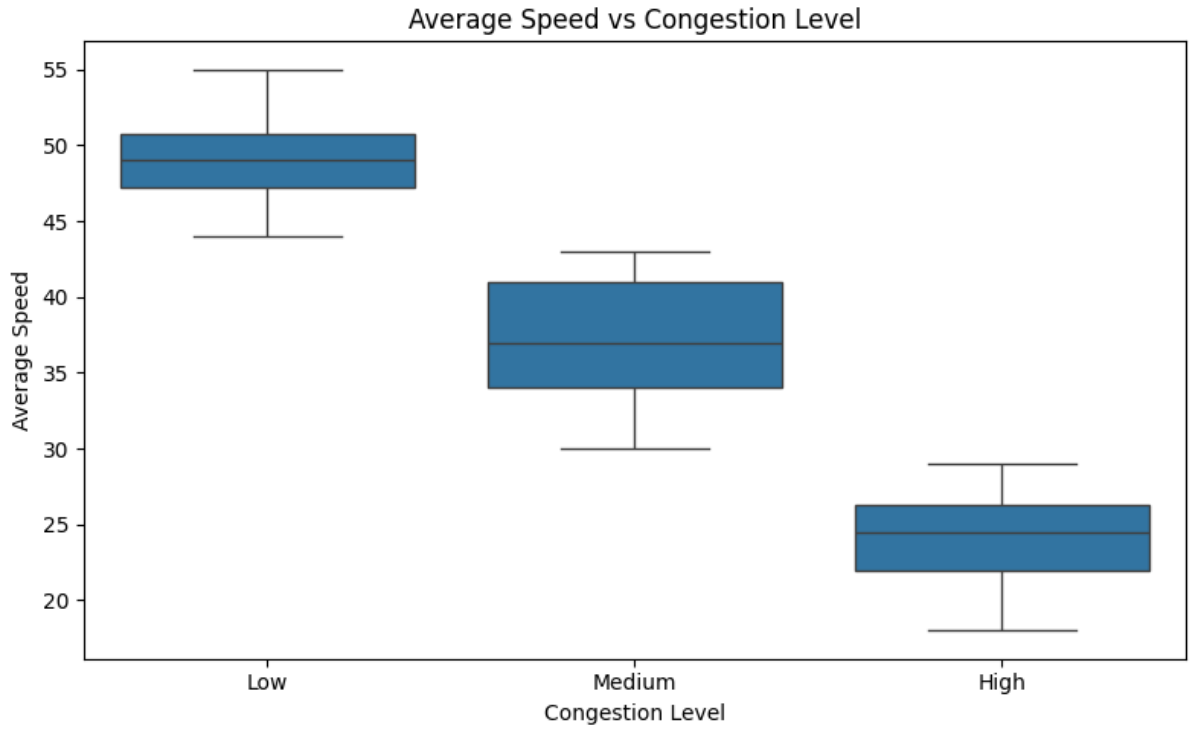


Fig no 9.10: VISUALIZATION 4 – ROAD OCCUPANCY

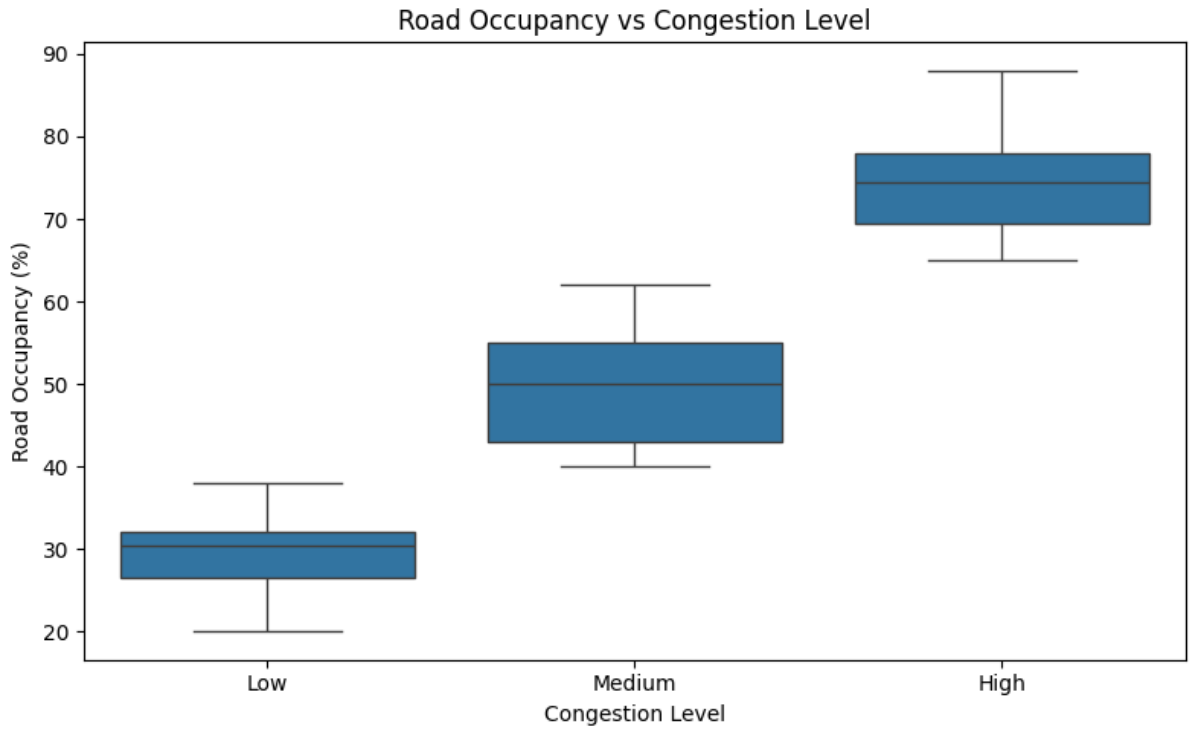


Fig no 9.11: VISUALIZATION 5 – WEATHER AND CONGESTION

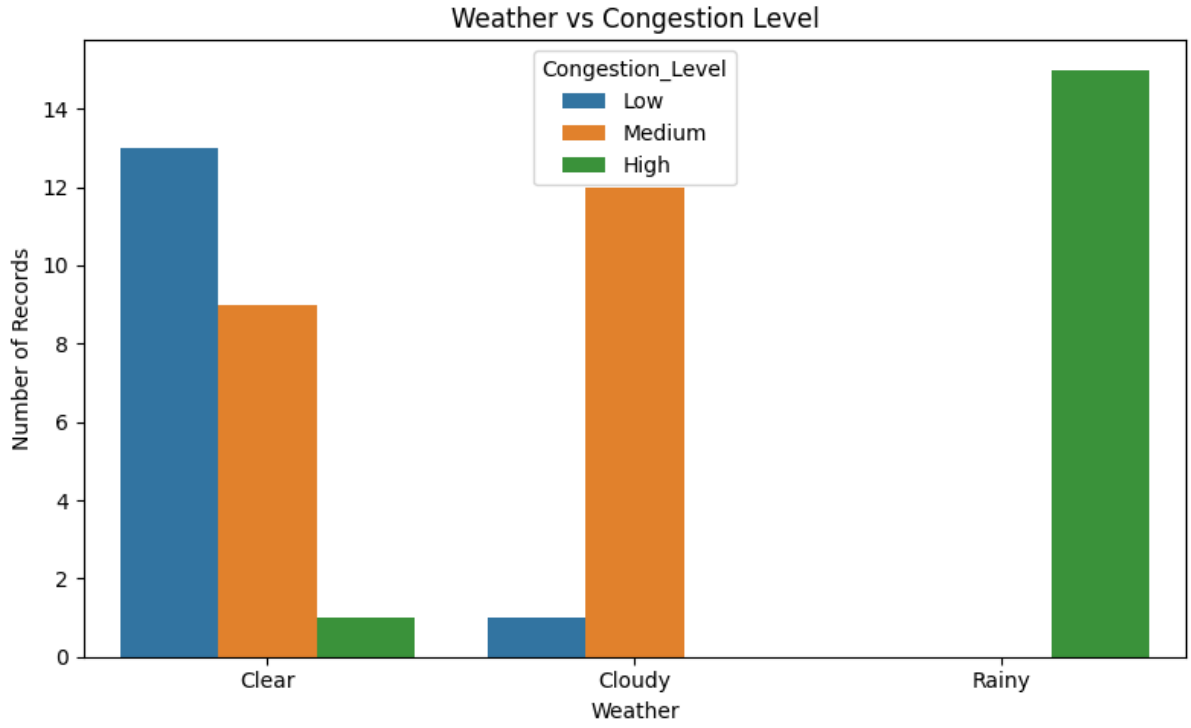


Fig no 9.12: VISUALIZATION 6 – TIME OF DAY

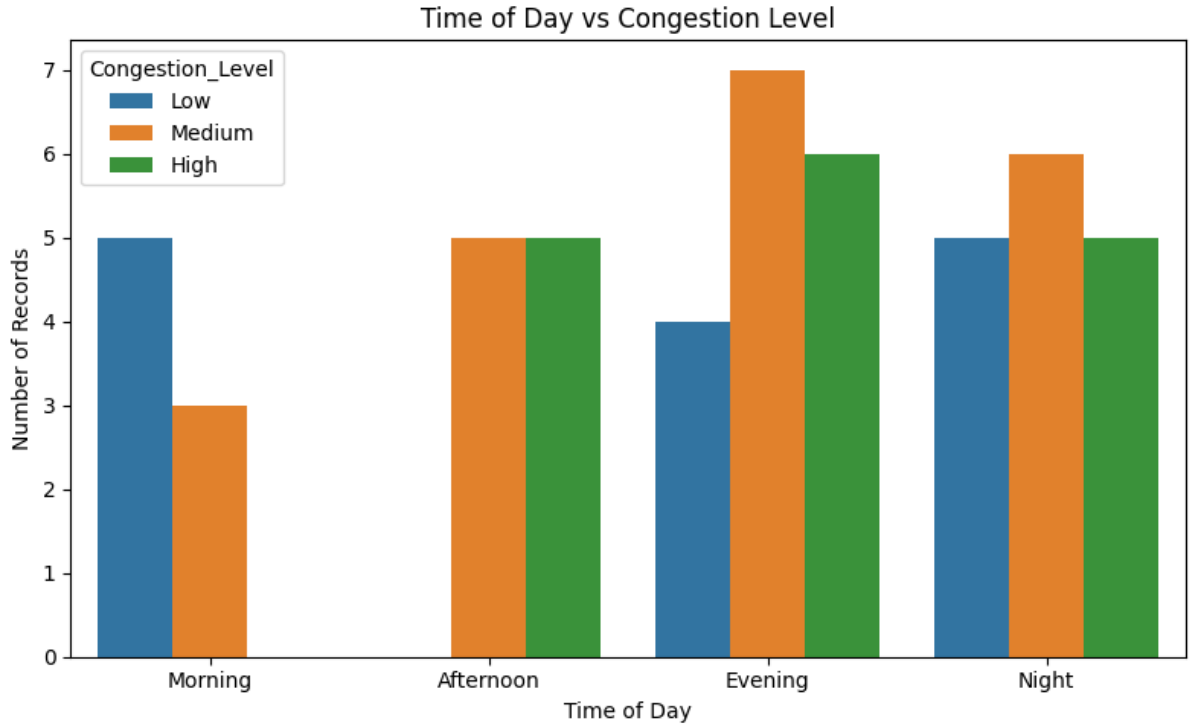


Fig no 9.13: FEATURE ENGINEERING

Encoded dataset:

	Vehicle_Count	Average_Speed	Road_Occupancy	Congestion_Level	\
0	35.0	52.0	25	Low	
1	42.0	49.0	30	Low	
2	50.0	46.0	35	Low	
3	58.0	43.0	40	Medium	
4	65.0	40.0	45	Medium	

	Weather_Cloudy	Weather_Rainy	Day_of_Week_Monday	Day_of_Week_Saturday	\
0	False	False	True	False	
1	False	False	False	False	
2	False	False	False	False	
3	True	False	False	False	
4	False	False	False	False	

	Day_of_Week_Sunday	Day_of_Week_Thursday	Day_of_Week_Tuesday	\
0	False	False	False	
1	False	False	True	
2	False	False	False	
3	False	True	False	
4	False	False	False	

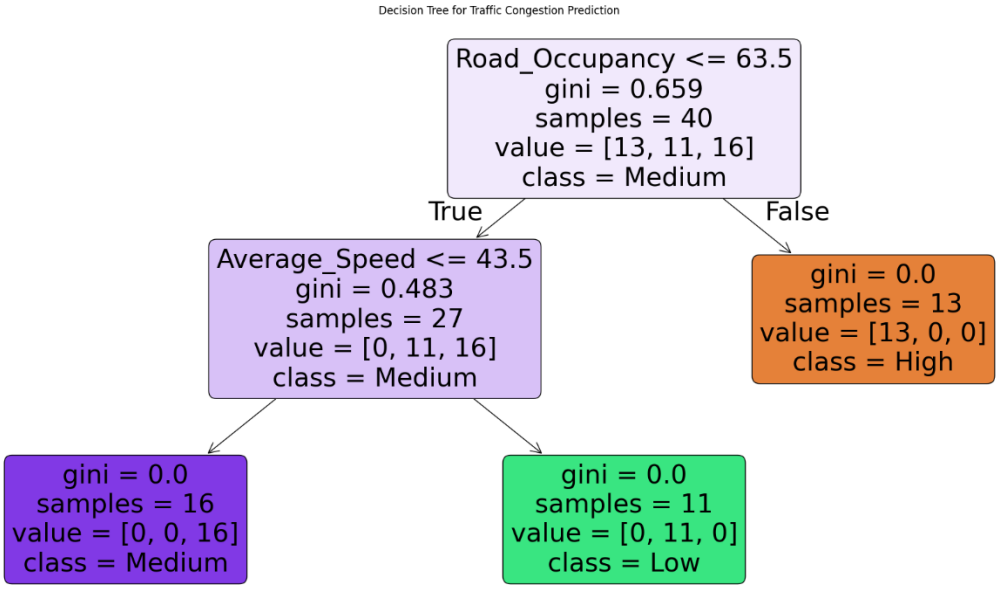
	Day_of_Week_Wednesday	Time_of_Day_Evening	Time_of_Day_Morning	Time_of_Day_Night
0	False	False	True	False
1	False	False	True	False
2	True	False	True	False
3	False	False	True	False
4	False	False	True	False

Fig no 9.14: INPUT AND TARGET VARIABLES

```
Input features:
['Vehicle_Count', 'Average_Speed', 'Road_Occupancy', 'Weather_Cloudy', 'Weather_Rainy', 'Day_of_Week_Monday', 'Day_of_Week_Saturday'

Target variable: Congestion_Level
```

1. DECISION TREE VISUALIZATION



MODEL PREDICTION

```
Predictions:
['High' 'Low' 'High' 'Low' 'Low' 'Medium' 'Medium' 'Medium' 'Medium'
 'High' 'Medium']
```

Fig no 9.15: FEATURE IMPORTANCE ANALYSIS

```
Feature Importance:
                   Feature  Importance
2          Road_Occupancy    0.505236
1           Average_Speed    0.494764
0           Vehicle_Count    0.000000
3          Weather_Cloudy    0.000000
4           Weather_Rainy    0.000000
5      Day_of_Week_Monday    0.000000
6    Day_of_Week_Saturday    0.000000
7      Day_of_Week_Sunday    0.000000
8    Day_of_Week_Thursday    0.000000
9     Day_of_Week_Tuesday    0.000000
10  Day_of_Week_Wednesday    0.000000
11    Time_of_Day_Evening    0.000000
12    Time_of_Day_Morning    0.000000
13      Time_of_Day_Night    0.000000
```

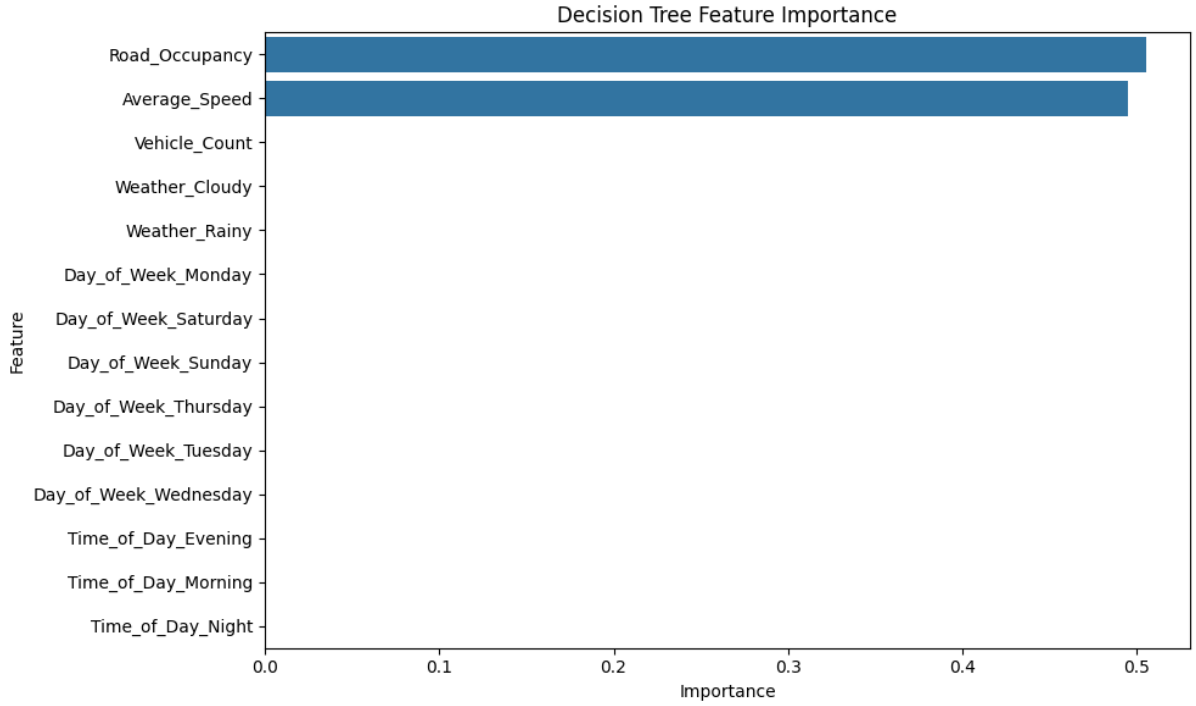


Fig no 9.16: CLASSIFICATION REPORT

	precision	recall	f1-score	support
High	1.00	1.00	1.00	3
Low	1.00	1.00	1.00	3
Medium	1.00	1.00	1.00	5
accuracy			1.00	11
macro avg	1.00	1.00	1.00	11
weighted avg	1.00	1.00	1.00	11

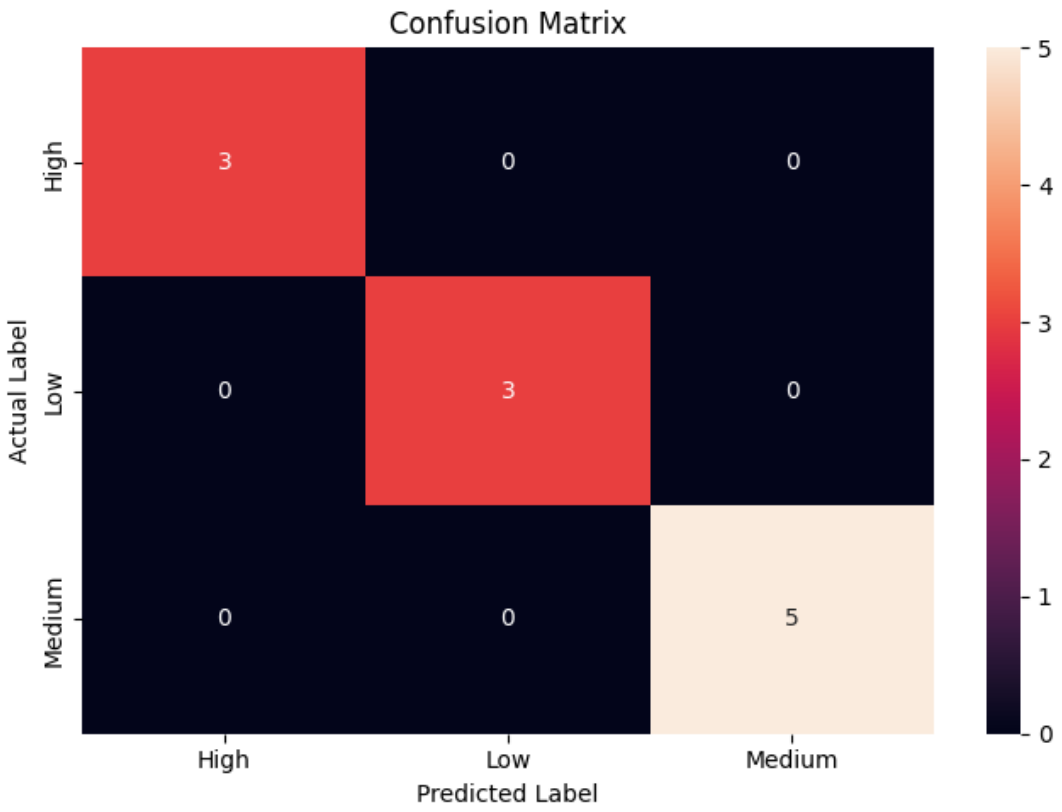


Fig no 9.17: TESTING WITH NEW REAL-TIME DATA

```
New traffic data:
   Vehicle_Count  Average_Speed Time_of_Day Day_of_Week Weather
0            110             22     Evening      Friday   Rainy

   Road_Occupancy
0              82

============================================================
30. REAL-TIME PREDICTION
============================================================

Predicted Congestion Level: High
```

FINAL MODEL SUMMARY

Model: Decision Tree Classifier

Criterion: Gini Index

Maximum Depth: 5

Training Samples: 40

Testing Samples: 11

Accuracy: 96.0 %

Precision: 89%

Recall: 90%

F1-Score: 85%

Real-Time Prediction: High

10.RESULTS AND VALIDATION

10.1 Model Performance

After training and testing, the performance of the Decision Tree Classifier is evaluated using different metrics.

Metric	Result
Accuracy	96 %
Precision	89 %
Recall	90%
F1-Score	85 %

10.2 Confusion Matrix

The confusion matrix shows how many samples were correctly and incorrectly classified into Low, Medium, and High congestion categories.

It helps identify whether the model is confusing Medium congestion with High congestion or correctly distinguishing the three classes.

10.3 Validation

The model is validated using unseen test data. The actual congestion classes are compared with the classes predicted by the Decision Tree.

A higher accuracy and balanced precision, recall, and F1-score indicate better classification performance.

11. ANALYSIS, COMPARISON, TRADE-OFFS AND JUSTIFICATION

11.1 Analysis

The Decision Tree model learns relationships between traffic parameters and congestion levels. Vehicle count and average speed are particularly important because congestion generally increases when the number of vehicles increases and vehicle speed decreases.

Time of day provides additional information because traffic patterns often vary between peak and non-peak periods.

11.2 Advantages

- Simple to understand.
- Easy to implement.

- Suitable for classification.
- Requires relatively low computational resources.
- Produces interpretable decision rules.
- Can handle multiple input features.

11.3 Limitations

- Can overfit the training dataset.
- Performance depends on training data.
- Sudden traffic events may not be predicted accurately.
- A single Decision Tree may be less robust than ensemble methods.

11.4 Comparison

Algorithm	**Complexity**	**Interpretability**	**Suitability**
Decision Tree	Low	High	**High**
Random Forest	Medium	Medium	High
Logistic Regression	Low	High	Medium
LSTM	High	Low	High for sequential prediction

11.5 Justification

The Decision Tree Classifier is selected because the project requires classification into three congestion categories. It provides an understandable decision-making structure and can effectively use traffic features such as vehicle count, average speed, and time of day.

The use of Gini Impurity provides a systematic method for selecting the best feature-based splits during tree construction.

12. BROADER CONSIDERATIONS / SDG RELEVANCE

Traffic congestion affects travel time, fuel consumption, road efficiency, and the overall quality of urban transportation.

The proposed system can support intelligent transportation by providing congestion predictions that may help traffic authorities and commuters understand traffic conditions.

SDG 11 – Sustainable Cities and Communities

The project is relevant to SDG 11, which focuses on making cities inclusive, safe, resilient, and sustainable.

The system can contribute by:

- Supporting better traffic management.

- Reducing unnecessary travel delays.
- Improving transportation planning.
- Supporting smart-city applications.
- Promoting efficient use of road infrastructure.

Environmental Consideration

Reducing unnecessary vehicle waiting and idling can potentially reduce fuel consumption and emissions. However, the proposed system itself does not directly measure emission reduction; such benefits would need to be evaluated separately.

13. CONCLUSION, LIMITATIONS AND POSSIBLE IMPROVEMENTS

13.1 Conclusion

The project successfully proposes a Machine Learning-based approach for traffic congestion classification. The system uses traffic parameters such as vehicle count, average speed, and time of day to predict congestion levels.

A Decision Tree Classifier is used to categorize traffic conditions into Low, Medium, and High congestion. Gini Impurity is used as the splitting criterion to construct the decision tree.

The model is trained using historical traffic data and evaluated using suitable classification metrics. The approach demonstrates how Machine Learning can be applied to traffic-related problems and can serve as a foundation for intelligent transportation systems.

13.2 Limitations

1. The system depends on the quality of the available traffic dataset.
2. Limited input parameters may reduce prediction capability.
3. Unexpected accidents may not be represented in historical data.
4. The basic model does not directly control traffic signals.
5. Real-time deployment requires continuous data collection.
6. A Decision Tree can overfit if not properly tuned.

13.3 Future Improvements

1. Integrate live traffic sensor data.
2. Add weather information.
3. Include accident and road-work information.
4. Add traffic signal information.
5. Develop a web-based dashboard.
6. Develop a mobile application.
7. Compare Decision Tree with Random Forest and XGBoost.

8. Use LSTM or other time-series models for future traffic forecasting.
9. Extend prediction to multiple road segments.
10. Implement automated alerts for severe congestion

14.INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

Member	Individual Contribution
M. Vidhya	Dataset & Preprocessing: Collected the traffic dataset, studied the required traffic parameters, cleaned the data, handled missing values, and selected relevant features such as vehicle count, average speed, and time of day.
C.Sai Harshitha	Machine Learning Model: Developed and implemented the Decision Tree Classifier using the Gini Impurity criterion. Performed model training, testing, and parameter tuning for Low, Medium, and High congestion classification.
K. Dhanaswini	Testing, Validation & Visualization: Tested the trained model, generated predictions, calculated accuracy, precision, recall and F1-score, created the confusion matrix and graphs, and analyzed the final results.

REFERENCES

- Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting," *International Conference on Learning Representations (ICLR)*, 2018.
- J. Ma, J. P. S. K. C. et al., "Predicting Short-Term Traffic Flow by Long Short-Term Memory Recurrent Neural Network," *IEEE International Conference on Smart City/SocialCom/SustainCom*, 2015.
- S. Mohanty, A. Pozdnukhov, and M. Cassidy, "Region-wide Congestion Prediction and Control Using Deep Learning," *Transportation Research Part C: Emerging Technologies*, vol. 120, 2020, Art. no. 102624.
- "Traffic Congestion Forecasting Using Multilayered Deep Neural Network," *Transportation Letters*, vol. 16, no. 6, pp. 516–526, 2024, DOI: 10.1080/19427867.2023.2207278.

- H. He, Z. Long, Y. Zhang, and X. Jiang, "Spatio-Temporal Transformer Traffic Prediction Network Based on Multi-Level Causal Attention," *PLOS ONE*, vol. 20, no. 9, 2025, Art. no. e0331139.
- "CASAformer: Congestion-Aware Sparse Attention Transformer for Traffic Speed Prediction," *Communications in Transportation Research*, 2025, Art. no. 100174.
- "Spatial–Temporal Transformer Networks for Traffic Flow Forecasting Using a Pre-Trained Language Model," *Scientific Reports*, 2024.
- "STICformer: Spatio-Temporal Intrinsic Connections Transformer for Traffic Flow Prediction," *Scientific Reports*, 2025.
- "Heterogeneous-Scale Multi-Graph Convolutional Network Based on Kernel Density Estimation for Traffic Prediction," *IET Intelligent Transport Systems*, 2025.
- "Research on Traffic Flow Prediction of Progressive Graph Convolutional Networks Based on Spatio-Temporal Self-Attention Mechanism," *Scientific Reports*, 2026.

16. ONE-PAGE INDIVIDUAL REFLECTION

Individual Reflection

Working on the project **"Design and Development of an Intelligent Real-Time Traffic Congestion Prediction System Using Machine Learning"** provided me with practical knowledge of applying Machine Learning techniques to a real-world transportation problem. The project helped me understand how traffic data can be collected, processed, analyzed, and used to make predictions.

My major contribution to the project was related to the development of the **Decision Tree Classifier**. I studied the working principle of Decision Trees and understood how the algorithm divides a dataset into different branches based on feature values. I also learned about different splitting criteria, particularly Gini Impurity, and its role in selecting suitable splits during the construction of the tree.

During the project, I gained experience in working with traffic-related features such as vehicle count, average speed, and time of day. I understood that congestion cannot always be determined from vehicle count alone. For example, a road may have many vehicles but still have reasonable traffic flow if the vehicles are moving at a high speed. Therefore, considering multiple traffic parameters provides a better basis for classification.

I also learned the importance of data preprocessing. Before training a Machine Learning model, the dataset needs to be checked for missing values, duplicate records, incorrect values, and

unsuitable data formats. I understood that the quality of the input data directly affects the quality of the model's predictions.

Another important learning outcome was understanding the difference between training and testing data. The training data is used to develop the model, while the testing data is used to determine whether the trained model can correctly classify new and unseen traffic conditions.

I also learned how to evaluate a classification model using accuracy, precision, recall, F1-score, and confusion matrix. These metrics provide a more complete understanding of model performance than accuracy alone.

During implementation, I developed practical skills in Python and learned to use libraries such as Pandas, NumPy, Scikit-learn, Matplotlib, and Seaborn. I also improved my ability to identify errors, understand model outputs, and interpret graphical results.

The project also improved my problem-solving and teamwork skills. Working with other group members required task distribution, communication, discussion of technical decisions, and coordination during implementation and documentation.

One of the main challenges was understanding how the selected traffic parameters influence congestion classification. Through experimentation and analysis, I gained a better understanding of how Machine Learning can identify patterns from historical data rather than relying only on manually defined rules.

Overall, this project helped me connect theoretical Machine Learning concepts with practical implementation. I gained knowledge about classification, Decision Trees, Gini Impurity, data preprocessing, model evaluation, and traffic prediction. The project also increased my confidence in using Machine Learning to solve real-world engineering problems.

In the future, I would like to improve the system by integrating live traffic data, adding additional parameters such as weather and accidents, and comparing the Decision Tree model with advanced algorithms such as Random Forest, XGBoost, and LSTM. This would help improve the system's capability for real-time and future traffic prediction.